Reviewer Pack — Minimal Order of Topological Field Theory States and Communi…

Entry 54a2fa05a8c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 13:24:40 UTC

Minimal Order of Topological Field Theory States and Communication Complexity Rank

Entry ID: `54a2fa05a8c4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 13:24:40 UTC

1. Conjecture

Field A (mathematical branch): Topological Field Theory (TFT) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every d-regular graph G, the minimal order of TFT states required to encode its associated communication complexity matrix is O(d^2 log n), where n is the number of variables in the graph.

Rationale (proposer's reasoning):

> Topological Field Theory provides a powerful framework for studying topological properties of spaces. By applying this theory to encode communication complexity matrices, we might uncover new structural insights into computational tasks that involve information exchange between parties.

Taxonomy category: `TFT_to_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c13c825091f71bad`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of generated d-regular graphs (n ≤ 40) have a TFT state order O(d^2 log n), and all graphs have a correlation coefficient between the TFT state order and O(d^2 log n) greater than or equal to 0.8, otherwise it is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Topological Field Theory' AND 'Communication Complexity' AND 'matrix rank' - 'd-regular graph' AND 'topological field theory' AND 'communication complexity' - O(d^2 log n) AND 'topological field theory states' AND 'communication complexity matrix'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges_added = set()

    while len(edges_added) < (n * d) // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)

        if u != v and (u, v) not in edges_added and (v, u) not in edges_added:
            graph[u].append(v)
            graph[v].append(u)
            edges_added.add((u, v))

    return graph

def communication_complexity_matrix(graph):
    n = len(graph)
    matrix = [[0] * n for _ in range(n)]

    for u in range(n):
        for v in range(u + 1, n):
            if v in graph[u]:
                matrix[u][v] = 1
                matrix[v][u] = 1

    return matrix
```

```
def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    augmented_matrix = [row[:] + [i] for i, row in enumerate(matrix)]

    def gaussian_elimination(A):
        rows, cols = len(A), len(A[0])
        for col in range(cols - 1):
            pivot_row = None
            for row in range(col, rows):
                if A[row][col] != 0:
                    pivot_row = row
                    break

            if pivot_row is None:
                continue

            A[pivot_row], A[col] = A[col], A[pivot_row]

            for row in range(rows):
                if row == col:
                    continue

                factor = -A[row][col] / A[col][col]
                for j in range(cols):
                    A[row][j] += factor * A[col][j]

        rank = 0
        for row in range(rows):
            if any(A[row][i] != 0 for i in range(cols - 1)):
                rank += 1

        return rank

    return gaussian_elimination(augmented_matrix)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        d = random.randint(2, min(n - 1, 5))
        graph = generate_d_regular_graph(n, d)
        matrix = communication_complexity_matrix(graph)

        if len(matrix) == 0 or len(matrix[0]) == 0:
            continue

        rank_value = rank(matrix)
        expected_bound = Fraction(d ** 2 * math.log(n), 1).limit_denominator()
```

```
        total_metric_value += abs(rank_value - expected_bound)
        instances_tested += 1

        if rank_value > expected_bound:
            conjecture_holds = False
            counterexample = f"Graph size {n}, degree {d}: Rank {rank_value} exceeds expected bour

    mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0

    return {
        "metric_name": "Rank of Communication Complexity Matrix",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b19dde6.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b19dde6.py", line 94, in run_trial
    graph = generate_d_regular_graph(n, d)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b19dde6.py", line 20, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing any data, which means that the pre-registered support conditions could not be evaluated. | next: Investigate and fix the error causing the test to crash, then rerun the test to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20636
2	preregistration	ollama_remote	glm4:latest	0	0	9953
3	novelty	ollama_remote	glm4:latest	0	0	8723
4	novelty	ollama_remote	glm4:latest	0	0	11922
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19328
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16227
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14419
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13960
9	judge	ollama_remote	glm4:latest	0	0	21610

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 136779 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/54a2fa05a8c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/54a2fa05a8c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/54a2fa05a8c4.tar.gz` (if generated)